15 動態規劃

Part IV Advanced Design and Analysis Techniques

本章定位

動態規劃的重點，是把「剩下要解什麼」定義清楚，再把同一個子問題的答案存下來。本章從木料切割開始，接著處理矩陣鏈、最長共同子序列與最佳二元搜尋樹。每個問題都要回答三件事：狀態記什麼、為什麼轉移不漏解，以及算完最佳值後如何找回具體答案。

前置知識

遞迴、最佳化與子問題重疊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 建立 rod cutting 與 LCS 的最佳子結構。
- 用維度與計算次序設計 DP 表。
- 推導 matrix-chain 與 optimal BST 的區間 DP。
- 由 choice 表完成重建，並判斷 DP 條件在陌生變形中是否仍成立。
- 15.1 Rod cutting
- 15.2 Matrix-chain multiplication
- 15.3 Elements of dynamic programming
- 15.4 Longest common subsequence
- 15.5 Optimal binary search trees

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

15.1 Rod cutting

有一根長度為n的木料，長度i的整段售價是p_i。每段長度必須是正整數，可以完全不切，但整根木料都必須賣出；本節先不計切割費用。我們要找的是總售價最高的切法。

先固定第一段的長度i：這一段收入p_i，剩下的n−i仍是同一種切割問題。令r[n]表示長度n能得到的最高收益，空木料的收益r[0]=0，因此

$$r[n] = \max_{1 \le i \le n} \{p_i + r[n-i]\}.$$

為什麼剩下的部分也要用最佳切法？ 假設一個最佳方案先切下長度i，但後面的切法不是長度n−i的最佳方案。把後半段換成更高收益的切法，第一段與總長度都不變，總收入卻提高了，與原方案最佳矛盾。

反過來，每個候選p_i+r[n−i]都對應一種合法切法，所以候選最大值不可能超過真正的最佳收益。前一段證明最佳方案也一定出現在候選裡；兩個方向合起來，才得到這個等式。

若每次都重算相同長度，呼叫數滿足$T(0) = 1$、$T(n) = 1 + \sum_{j=0}^{n-1} T(j) = 2^n$。這裡T數的是函式呼叫，不是收益。把每個長度只算一次後，轉移總數降為$\sum_{j=1}^{n} j = \Theta(n^2)$。

由短到長填表，再保存第一段的選擇

偽碼用p[1]表示長度1的售價，依長度由短到長計算。choice[length]另外保存最佳方案的第一段長度，讓我們最後能重建切法。非空木料的初始收益要設為−∞：即使所有售價為負，題目仍要求整根賣出，不能用0偷偷代表丟棄。

```
r[0] = 0
for length = 1..n:
    r[length] = -∞
    for first = 1..length:
        value = p[first] + r[length-first]
        if value > r[length]:
            r[length] = value
            choice[length] = first
pieces = []
while n > 0:
    append choice[n] to pieces
    n = n-choice[n]
```

進入某個length時，更短的長度都已算好。內圈把每一種第一段長度都比較完，才確定r[length]與choice[length]。重建時，每取出一段，剩餘長度就至少減1；到0便完成，最多輸出n段。

Rod Cutting 的完整表與切割重建

取 `p=[1,5,8,9]`，`choice[n]` 記錄取得最大值的第一段長度：

n	候選 p_i+r[n-i]	r[n]	choice[n]
0	空	0	0
1	1	1	1
2	5, 1+1=2	5	2
3	8, 1+5=6, 5+1=6	8	3
4	9, 1+8=9, 5+5=10, 8+1=9	10	2

回溯由 n=4 開始：`choice[4]=2`，收益加上 `p_2=5`，剩長 2；`choice[2]=2`，再加 5，剩 0。故最佳切法 `2+2`，總收益 10。表已算完才回溯，不能邊猜切點邊改動已定的最優值。

若每刀收固定費用c，只在切完第一段後仍有剩料時扣c，仍能用「剩餘長度」當狀態。若改成最多只能切成k段，未來能否再切就取決於已用段數，狀態必須多記這項資訊。判斷是否增加維度，要看它會不會改變後續的合法選擇。

15.2 Matrix-chain multiplication

矩陣的先後順序固定；我們只能選括號，不能重排矩陣。設A_i的維度是$p_{i-1} \times p_i$。若最後一次乘法把區間切在k，左側結果為$p_{i-1} \times p_k$，右側為$p_k \times p_j$，所以這個切點的總成本是

$$c_k = m[i,k] + m[k+1,j] + p_{i-1}p_k p_j.$$

前兩項是左右子鏈已付出的成本，最後一項才是本次相乘新增的純量乘法次數。枚舉$k = i, \dots, j-1$後取最小：$m[i,j] = \min_k c_k$。單一矩陣不需要乘法，所以$m[i,i] = 0$。

固定最後切點k後，左右結果的維度已經確定。如果某一側還能用更便宜的括號化，就能替換那一側，而不改變最後一次乘法的維度與成本；整體也會變得更便宜。因此最佳方案的左右子鏈必須各自最佳。另一方面，每一種完整括號化都有最後切點，枚舉所有k便不會漏解。

依區間長度l=j−i+1遞增填表，計算某一格時，左右子鏈都比它短，答案已經存在。split[i,j]保存達到最低成本的切點。共有Θ(n²)個區間，每格最多比較O(n)個切點，因此時間上界為O(n³)。

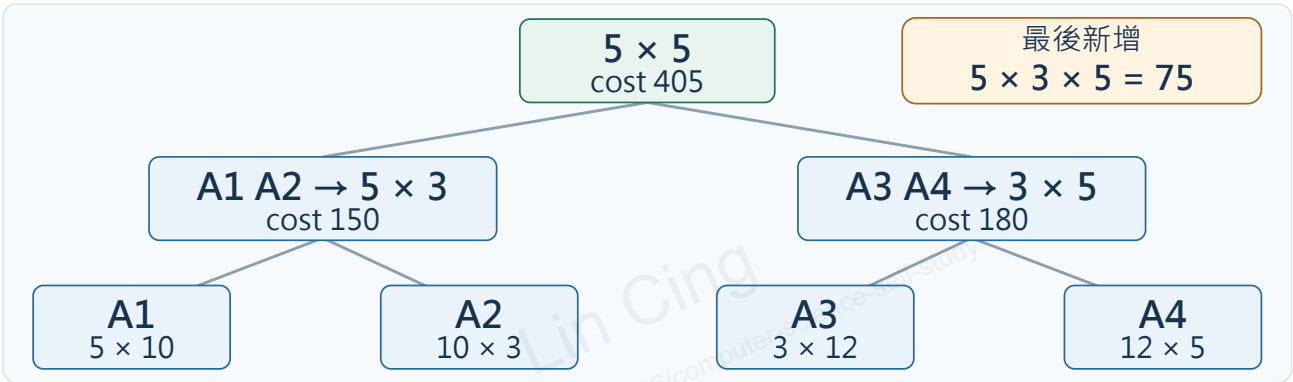
這個版本實際比較的切點總數是$\sum_{l=2}^{n}(n-l+1)(l-1) = n(n^2-1)/6$。每個候選只做常數次讀表與算術，所以時間是$\Theta(n^3)$，不只有一個上界。

Matrix-Chain 的填表與括號輸出

```
for i = 1..n: m[i,i] = 0
for length = 2..n:
    for i = 1..n-length+1:
        j = i+length-1
        m[i,j] = +∞
        for k = i..j-1:
            c = m[i,k]+m[k+1,j]+p[i-1]*p[k]*p[j]
            if c < m[i,j]: m[i,j], split[i,j] = c, k
emit(i,j):
    if i == j: print A_i; return
    k = split[i,j]
    print '('; emit(i,k); emit(k+1,j); print ')'
```

每項轉移皆讀取嚴格較短的區間，所以 length 的順序合法。重建樹有 n 葉、n−1 個內節點，輸出括號需 Θ(n) 次節點操作；不含實際矩陣元素的乘法。

Matrix-Chain 的填表與括號輸出 （續）



150 + 180 + 75 = 405；累計成本不等於本次乘法成本

Matrix-Chain 的完整表與括號重建

取 `p=[5,10,3,12,5]`，A1.A4 的 m 上三角如下：

j-i+1	值
1	m11=m22=m33=m44=0
2	m12=150、m23=360、m34=180
3	m13=330、m24=330
4	m14=405

例如 `m24=min(0+180+10*3*5, 360+0+10*12*5)=330`，split[2,4]=2；`m14=min(0+330+5*10*5, 150+180+5*3*5, 330+0+5*12*5)=405`，split[1,4]=2。

回溯 `split[1,4]=2`，得到 `(A1A2)` 與 `(A3A4)`；兩段各以 `split[1,2]=1`、`split[3,4]=3` 展開，最終 `((A1A2)(A3A4))`。

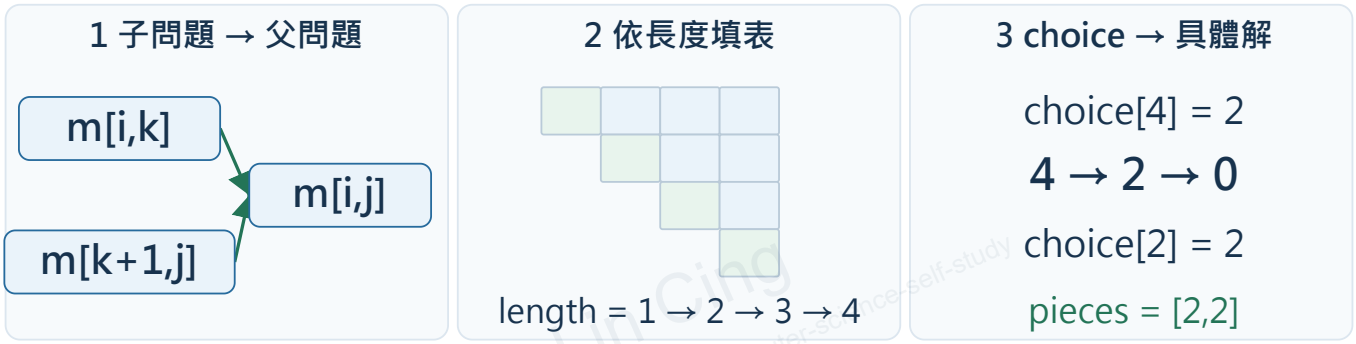
15.3 Elements of dynamic programming

狀態必須保留後續決策需要的資訊。替一個大問題命名還不夠：如果兩種情況被放進同一格，卻有不同的合法後續選擇，這一格就無法代表它們。本章採用可以依序填表的DP；證明時要交代以下四件事。

1. 最優解去掉第一個決策後，剩下的是對應子問題的最優解。
2. 每個原問題候選決策都能由已定義的子狀態組合，且沒有漏掉合法解。
3. 基底值正確，且依賴圖無環。
4. 拓撲次序保證填值時所有先決狀態已經算好。

最佳子結構說明「為什麼可以用子問題的最佳解組合」，重疊子問題則說明「為什麼存表能省下重算」。兩者的工作不同。若狀態依賴有環，不能直接套本章的一次填表流程。要重建答案，可以另存選擇表，也可以回頭檢查完整值表；只剩最後一個最佳值，通常不足以找回整個方案。

15.3 Elements of dynamic programming （續）



總時間 = 狀態處理成本 + 所有轉移成本

狀態圖決定快取的成本

rod 的依賴圖有 n+1 個狀態，長度 j 向 0..j−1 各有一個依賴邊，共 n(n+1)/2 條。若每狀態與每條邊只處理常數次，時間是 Θ(V+E)=Θ(n^2)。

自頂向下 memo 在第一次計算狀態時展開全部候選，之後立即返回已存值。visited 必須獨立於數值，例如以 None 區分未算；不能把合法的負收益當作「未計算」。由小到大填表沒有遞迴堆疊，memo 最多有 n 層呼叫堆疊，兩者另存表均 Θ(n)。

最長簡單路徑說明了狀態遺漏資訊的後果。兩段各自最長的簡單路徑u→v與v→w，可能都經過頂點z；拼起來就重複z，不再簡單。只記起終點無法排除這種情況。加入已訪問集合S可以修正，但狀態數會變成指數；若輸入圖本來就是DAG，則可直接依拓撲順序求最長路徑。

15.4 Longest common subsequence

令$c[i,j]$表示兩個前綴$X[1..i]$、$Y[1..j]$的最長共同子序列長度。子序列保留字元順序，但可以跳過字元，不要求連續。c表先回答長度；實際字串稍後再回溯。任一前綴為空時，長度為0。

先看兩個末字元相同的情況，記共同字元為c。如果一個LCS不以c結尾，就不會使用任一輸入的末位置，於是還能在後面接c，與「最長」矛盾。若它以c結尾，可以把最後一次配對放到這兩個末位置；拿掉c後，剩下的字元都落在兩個較短前綴裡。

若兩個末字元不同，任何共同子序列都不可能同時使用這兩個末位置：兩者一旦使用，都必須是共同序列的最後一個字元，卻又互不相同。因此至少有一個末位置未被使用，只需比較「去掉X末字元」與「去掉Y末字元」兩個子問題。

`c[i,j]=max(c[i-1,j], c[i,j-1])`。

任一共同子序列都落在其中一個分支，而每個分支的解也都是原問題的合法子序列，所以取兩者最大值正確。每格只需常數次比較；兩個字串皆非空時，完整填表時間為Θ(mn)。空字串可直接回傳長度0。

LCS 的兩向界與完整實作

末字元相同時，刪去最後一次配對，得到的長度至多是$c[i-1,j-1]$；反過來，任何較短前綴的LCS都能接上這個共同末字元。因此$c[i,j]=c[i-1,j-1]+1$。這裡需要的是一條正確的最佳配對路徑，不必聲稱所有最佳配對都經過同一格。

```
c[0,0..n] = 0; c[0..m,0] = 0
for i = 1..m:
    for j = 1..n:
        if X[i] == Y[j]: c[i,j] = c[i-1,j-1]+1
        else: c[i,j] = max(c[i-1,j],c[i,j-1])
i=m; j=n; result=[]
while i>0 and j>0:
    if X[i]==Y[j]: append X[i] to result; i--; j--
    elif c[i-1,j]>=c[i,j-1]: i--
    else: j--
reverse result
```

完整c表已經足以判斷回溯方向，不一定要另存方向表。若只求長度，可沿較短字串保留兩列，空間為O(min(m,n)+1)。不過，上面這段回溯會讀取先前各列，不能直接把完整表丟掉後照用。

LCS 的完整表與回溯

X=ABCBDAB、Y=BDCABA，本例 c[7,6]=4。加上空前綴後的 c 表為：

```
      ε B D C A B A
ε     0 0 0 0 0 0 0
A     0 0 0 0 1 1 1
B     0 1 1 1 1 2 2
C     0 1 1 2 2 2 2
B     0 1 1 2 2 3 3
D     0 1 2 2 2 3 3
A     0 1 2 2 3 3 4
B     0 1 2 2 3 4 4
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

LCS 的完整表與回溯 （續）

回溯時，表格中的「上」代表 i 減一，「左」代表 j 減一：

當前格	動作	下一格	收集的反序字元
(7,6)	上，兩候選同為 4	(6,6)	空
(6,6)	A=A，左上	(5,5)	A
(5,5)	D≠B，上方值 3 較大	(4,5)	A
(4,5)	B=B，左上	(3,4)	AB
(3,4)	C≠A，左方值 2 較大	(3,3)	AB
(3,3)	C=C，左上	(2,2)	ABC
(2,2)	B≠D，左方值 1 較大	(2,1)	ABC
(2,1)	B=B，左上	(1,0)	ABCB

反向輸出 BCBA。每步至少減一個索引，故回溯 O(m+n)，即使原表有 mn 格也不必掃完。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

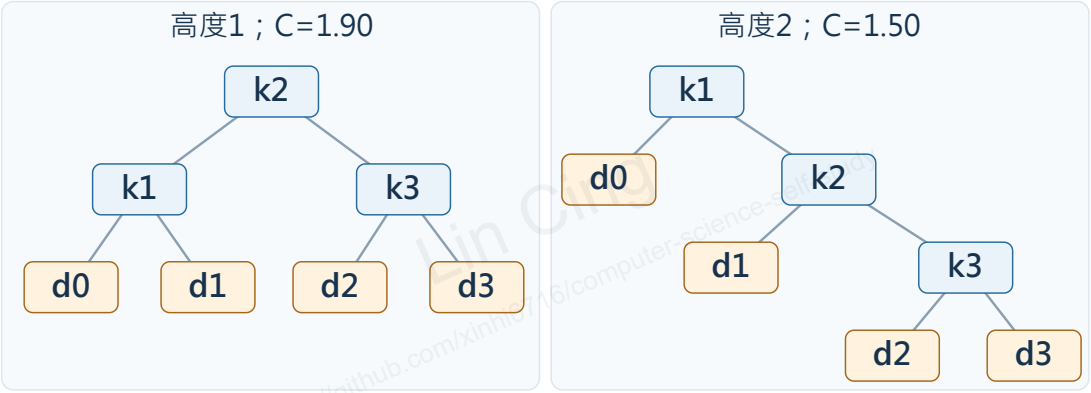
LCS 的完整表與回溯 (續)

i ↓ / j →	0 ε	1 B	2 D	3 C	4 A	5 B	6 A
0 ε	0	0	0	0	0	0	0
1 A	0	0	0	0	1	1	1
2 B	0	1	1	1	1	2	2
3 C	0	1	1	2	2	2	2
4 B	0	1	1	2	2	3	3
5 D	0	1	2	2	2	3	3
6 A	0	1	2	2	3	3	4
7 B	0	1	2	2	3	4	**4**

綠箭頭：回溯；橘邊：取字元；A,B,C,B 反轉 → BCBA

例A：樹高較小，不代表加權搜尋比較較少

查詢機率為：p=(0.50,0.10,0.05)，q=(0.15,0.10,0.05,0.05)。根深度0；高度只算實鍵間的邊；C只計實鍵比較。



C：只計實鍵比較；dummy 不再多算一次

例A：每一種成功與失敗查詢都要加權

成功搜尋比較數為鍵深度加1；失敗搜尋比較數為dummy深度，**不多算一次dummy檢查**。

查詢	機率	平衡樹：比較數 × 機率	右斜樹：比較數 × 機率
k1	0.50	2 × 0.50 = 1.00	1 × 0.50 = 0.50
k2	0.10	1 × 0.10 = 0.10	2 × 0.10 = 0.20
k3	0.05	2 × 0.05 = 0.10	3 × 0.05 = 0.15
d0	0.15	2 × 0.15 = 0.30	1 × 0.15 = 0.15
d1	0.10	2 × 0.10 = 0.20	2 × 0.10 = 0.20
d2	0.05	2 × 0.05 = 0.10	3 × 0.05 = 0.15
d3	0.05	2 × 0.05 = 0.10	3 × 0.05 = 0.15
總計	1.00	**C = 1.90**	**C = 1.50**

成功成本：1.20／0.85；失敗成本：0.70／0.65。查詢常落在k1或d0，右斜樹讓這些高機率結果較早結束，足以補償其他結果多做的比較。

15.5 Optimal binary search trees

給定互異且已排序的鍵k1<k2<⋯<kn，我們要選一棵BST，讓指定查詢分布下的平均搜尋成本最低。鍵的順序固定，能改的是誰當根、左右子樹怎麼安排。成功查詢與失敗查詢都必須計入。

查到鍵k_i的機率為p_i。找不到的查詢按排序間隙分成$d_0,\dots,d_n$，機率為$q_0,\dots,q_n$：例如d_0在最小鍵之前，d_i在相鄰鍵k_i,k_{i+1}之間。這些虛葉（dummy leaves）代表搜尋停止的空位置，不是一筆要比較的真實資料。所有p與q的總和為1。先定義把虛葉步驟也計入的擴充成本E：

$$E(T)=\sum_{r=1}^{n} p_r(\operatorname{depth}_T(k_r)+1)+\sum_{r=0}^{n} q_r(\operatorname{depth}_T(d_r)+1).$$

令$e[i,j]$為鍵$k_i..k_j$這段區間的最低擴充成本；$w[i,j]$是該區間全部成功鍵與失敗間隙的機率總和。即使沒有任何鍵，仍有一個失敗間隙，所以空段的初始化是

$$e[i,i-1]=w[i,i-1]=q_{i-1}.$$

如果選k_r當根，兩側子樹的每種查詢結果都要先經過這個根，深度因此各增加1。左右子問題的既有成本可以保留，再加上整段機率總和w[i,j]：

$$e[i,j]=\min_{i\le r\le j}\{e[i,r-1]+e[r+1,j]+w[i,j]\}.$$

15.5 Optimal binary search trees （續）

固定根後，兩側區間的鍵與間隙都已確定。若其中一側不是該區間的最佳樹，換成成本更低的同區間子樹，就能降低整體成本，與最佳性矛盾。每一棵合法BST都有某個根，因此比較所有根及其最佳子樹便涵蓋全部樹形。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一棵樹，兩種成本只差失敗機率總和

C(T)只數實際的鍵比較；E(T)還把失敗搜尋抵達虛葉的那一步算一次。這是兩種計費模型，不代表實作真的會與一筆虛構資料比較。接下來的區間表e採用E型成本。

$$C(T) = \sum_{r=1}^{n} p_r(\mathrm{depth}_T(k_r) + 1) + \sum_{r=0}^{n} q_r \,\mathrm{depth}_T(d_r).$$

$$E(T) = C(T) + \sum_{r=0}^{n} q_r.$$

例A的差為0.35：平衡樹E=2.25，右斜樹E=1.85。這是與樹形無關的常數，**最優樹不變，報出的成本卻不同**。

沒有鍵的區間，實鍵比較成本C[i,i−1]=0，擴充成本則為e[i,i−1]=q[i−1]。這不表示C忽略失敗查詢：空段掛到父節點下時，仍要比較父的鍵，這一步由父區間的w計入。使用哪種模型，初始化與後續加總就要一致。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

跨科對照：先確認搜尋模型，再比較答案

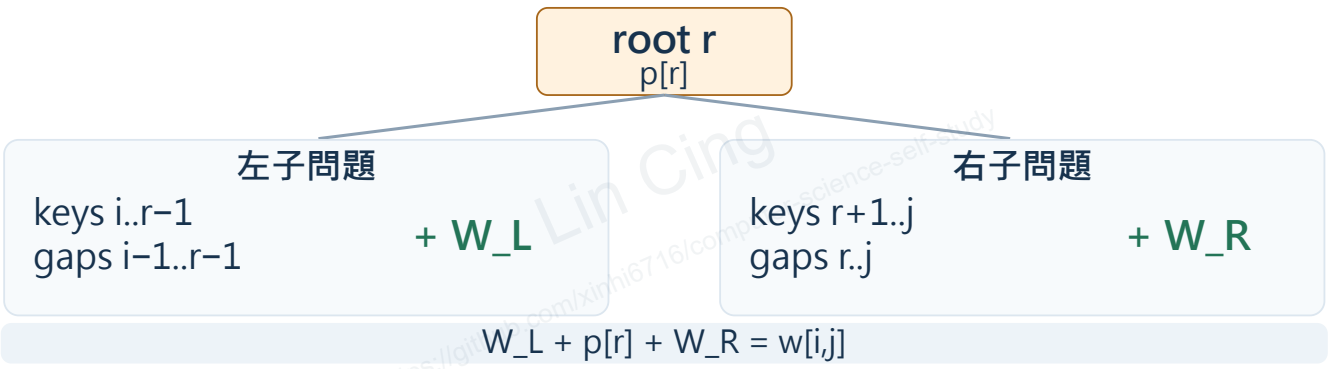
資料結構第10章先用「只有成功查詢」建立遞迴；本章依CLRS §15.5，將成功鍵與失敗間隙一起納入。兩者解的問題可能不同，不能只比較表格最後一個數字。

要確認的約定	只有成功查詢	本章的完整查詢模型
輸入權重	p；此模型下總和為1	p與q；兩者合計為1
空區間的成本	0	擴充成本E為q；實鍵成本C為0
每格新增成本	區間成功機率總和	區間成功與失敗機率總和w

q不全為0時，把q刪掉再將p正規化，會改變問題，最佳樹也可能改變。 相反地，在相同p、q下由E換成C，只扣掉常數Σq，最佳樹不變。讀其他教材時，也要核對區間端點是否包含、空樹初值與root欄位的含義。

為何加整段w，而不是只加根的p？

$$w[i,j]=\sum_{t=i}^{j} p_t+\sum_{t=i-1}^{j} q_t=W_L+p_r+W_R.$$



W_L, W_R是左右子樹機率總和；各查詢下降一層，增加W_L+W_R，根命中增加p_r。**機率沿用全域值，不將子區間重新正規化。**

Optimal BST 的填表及成本慣例

```
for i = 1..n+1:
    e[i,i-1] = w[i,i-1] = q[i-1]
for length = 1..n:
    for i = 1..n-length+1:
        j = i+length-1
        w[i,j] = w[i,j-1]+p[j]+q[j]
        e[i,j] = +∞
        for r = i..j:
            c = e[i,r-1]+e[r+1,j]+w[i,j]
            if c < e[i,j]: e[i,j], root[i,j] = c, r
```

偽碼用長度控制計算順序。每個根r都讀取已完成的左、右區間，root[i,j]存的是根的索引r，不是成本。例B的最低擴充成本為2.10；若只數實鍵比較，扣除全部失敗機率0.40後是1.70，最佳樹形不變。

重建時，先檢查區間是否為空：j<i就放入失敗間隙d[i−1]；否則取r=root[i,j]，建立鍵k_r，再遞迴處理[i,r−1]與[r+1,j]。把空區間也交代清楚，才能重建完整的成功鍵與失敗間隙。

例B：Optimal BST 的完整表與樹重建

取三鍵 p=(0.20,0.30,0.10)，dummy q=(0.10,0.05,0.15,0.10)。先算：

區間	w	e	最優根
[1,1]	0.35	0.50	k1
[2,2]	0.50	0.70	k2
[3,3]	0.35	0.60	k3
[1,2]	0.80	1.45	k2
[2,3]	0.70	1.35	k2
[1,3]	1.00	2.10	k2

例如 `e[1,2]` 以 k2 為根時，成本 `e[1,1]+e[3,2]+w[1,2]=0.50+0.15+0.80=1.45`。root[1,3]=2；左子問題 root[1,1]=1，右 root[3,3]=3，故樹根 k2，左右 child 分別 k1、k3，dummy 依 BST 區間掛放。

例B：Optimal BST 的完整表與樹重建 （續）

以長度填表，e、w、root 各 Theta(n^2) 格；每格枚舉最多 n 個根，共 Theta(n^3)。本章偽碼採這個已證明的版本。

候選根總數為 Σ_(l=1)^n (n−l+1)l=n(n+1)(n+2)/6。w 每格以 w[i,j−1]+p_j+q_j 常數時間更新，若在每個候選根內重掃整個機率區間，會多乘一個 n，不能沿用本分析。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

例B：逐一比較根，讀取正確的子問題

e[i,j]：空段到長區間

i / j	0	1	2	3
1	0.10	0.50	1.45	**2.10**
2		0.05	0.70	1.35
3			0.15	0.60
4				0.10

候選根；w[1,3]=1

r=1
0.10 + 1.35 + 1 = 2.45

r=2
0.50 + 0.60 + 1 = 2.10

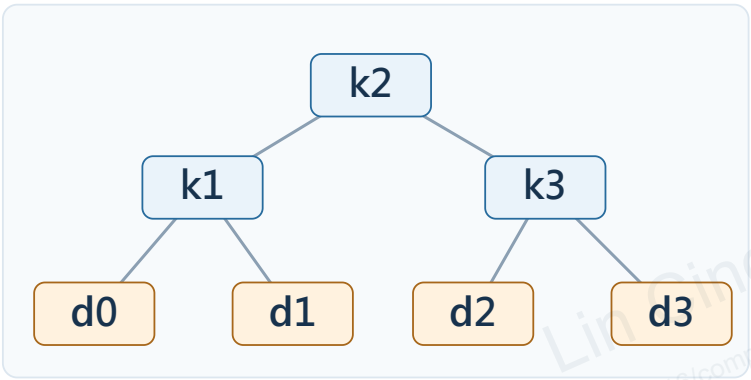
r=3
1.45 + 0.10 + 1 = 2.55

綠格：r=2 讀取的子問題；橘格：本次結果

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

綠格是選r=2時讀取的e[1,1]、e[3,3]。它們來自較短區間；橘格e[1,3]只在三個根都比較後確定為2.10。空段的q不可漏掉。

例B：從root表回到實際搜尋樹



例B：含 dummy 檢查

k2: 0.30 × 1 = 0.30

k1,k3: 0.30 × 2 = 0.60

d0..d3: 0.40 × 3 = 1.20

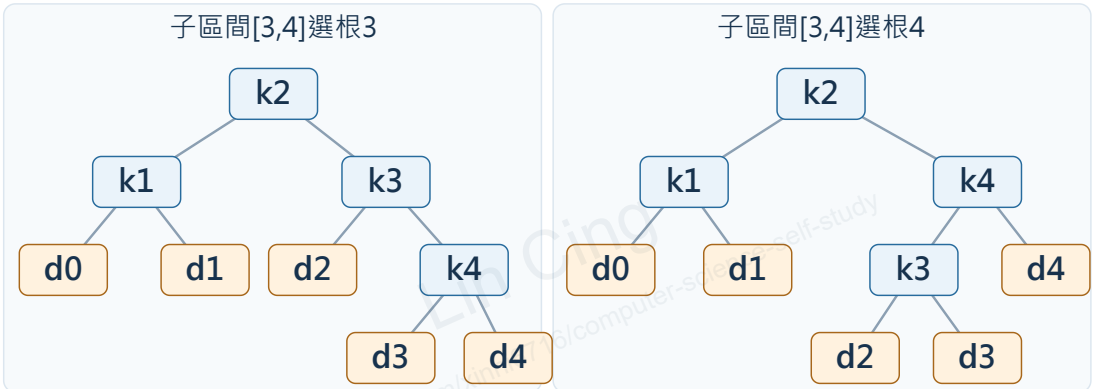
E = 2.10

C = E − 0.40 = 1.70

d0 | k1 | d1 | k2 | d2 | k3 | d3

例C：整數權重與兩棵同樣最優的樹

P=(3,3,1,1)、Q=(2,3,1,1,1)是權重，總和16，失敗權重和8。右子區間[3,4]選3或4同樣最優。



兩棵樹：S=32；除以總權重16後，C=2，E=2.50

序列對齊：配對、錯配與空位各有分數

全域序列比對允許 match、mismatch 與 gap，最後一欄可能是：

- `(x_i,y_j)` 配對，得分 match 或 mismatch；
- `x_i` 配 gap；
- `y_j` 配 gap。

若 `F[i,j]` 是前綴最高分，三種情形給出

`F[i,j]=max(F[i-1,j-1]+score(x_i,y_j), F[i-1,j]+gap, F[i,j-1]+gap)`。

這仍是前綴DP，但每次要比較三種最後一步。對X=ACGTA、Y=AGCAC，設定相同字元+3、不同字元−2、每個空位−3，最佳分數為1，可重建ACGTA-與A-GCAC。空位表示其中一側缺少字元，並不是原序列中的字元；每個空位都要付分數，不能只比較同一位置的兩個字元。

無限供應背包：先說清楚重量是恰好還是至多

每種物品可以取任意多次，物品i的重量為s_i、價值為v_i。令f[c]表示「重量恰好為c」時能得到的最高價值。這個「恰好」要先說清楚；無法湊出的重量不是價值0，而是不可行。

`f[0]=0`，`f[c]=max_{i:s_i<=c}(f[c-s_i]+v_i)`。

若容量只是上限K，答案是所有0≤c≤K的f[c]最大值；若要求恰好裝滿，才只看f[K]。把「未使用的一單位容量」也當成轉移，則是在定義另一個「重量至多c」的狀態，不能與現在的表義混用。

取物品A=(重量3,價值5)、B=(4,7)、C=(7,12)，容量上限K=13。以恰好重量定義的表為：

`f[0..13]=[0,-∞,-∞,5,7,-∞,10,12,14,15,17,19,21,22]`。

最高價值22出現在重量13，可選A+A+A+B。只看價值重量比會先取B，三件B價值21，反而少1。這個例子否定一般的比值貪婪規則；DP則把每種可行的最後一件物品都比較過。

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 15。

Cormen et al., *Introduction to Algorithms*, 3rd ed., §§15.1-15.5.

Lin Cing，個人資料結構手寫筆記：OBST加權搜尋成本、狀態區間與遞迴式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study